

## **Relatório do Projeto de Cloud Computing 2025/2026**

### **Introdução**

O presente relatório descreve o trabalho desenvolvido no âmbito da unidade curricular de Cloud Computing do curso de Engenharia Informática. O principal objetivo deste projeto consistiu na conceção e implementação de um sistema backend baseado em serviços da plataforma Microsoft Azure, de forma a compreender como a computação em nuvem permite criar aplicações escaláveis, rápidas e de elevada disponibilidade.

O sistema desenvolvido tem como propósito a gestão de coleções de Lego Sets e a realização de leilões entre utilizadores. Cada utilizador possui uma coleção de Lego Sets e pode colocar à venda um dos seus conjuntos, permitindo que outros licitem através de leilões. Além das funcionalidades de compra e venda, o sistema suporta o registo de utilizadores, o carregamento de imagens e vídeos, e a publicação de comentários associados a cada Lego Set.

A aplicação foi implementada segundo uma arquitetura modular e distribuída, tirando partido de vários serviços fornecidos pela plataforma Azure, nomeadamente o App Service, o Cosmos DB, o Blob Storage, o Redis Cache e as Azure Functions. Adicionalmente, foi integrada uma camada de pesquisa inteligente através do serviço Azure Cognitive Search, que permite efetuar pesquisa avançada sobre os comentários e descrições dos Lego Sets, devolvendo uma lista de conjuntos relacionados com a pesquisa efetuada pelo utilizador. O desenvolvimento teve como base os princípios de escalabilidade horizontal, eficiência no processamento de pedidos e tolerância a falhas.

### **Design do Sistema**

O sistema foi estruturado de forma a garantir a separação de responsabilidades entre os diferentes componentes e a facilitar a sua manutenção e evolução. A arquitetura segue um modelo em camadas, sendo cada uma responsável por uma parte distinta da aplicação.

A camada de dados define as entidades principais do sistema: User, LegoSet, Comment, Auction e Media. Cada uma destas classes possui um Data Access Object (DAO) correspondente, que facilita a conversão de objetos Java em documentos do Cosmos DB.

A camada de persistência, representada pela classe CosmosDBLayer, é responsável pela comunicação direta com o Azure Cosmos DB, assegurando as operações de criação, leitura, atualização e eliminação de dados. Esta camada também trata da sincronização com a cache Redis, eliminando ou atualizando as entradas em cache quando ocorrem alterações nos dados persistentes. O Cosmos DB foi ainda configurado com geo-replicação automática, garantindo redundância e disponibilidade em várias regiões sem necessidade de configuração manual.

A camada de cache foi implementada através do serviço Azure Cache for Redis, que armazena temporariamente dados frequentemente acedidos, como sessões de utilizadores, leilões ativos e listas de Lego Sets. A utilização desta camada reduz o

número de acessos diretos ao Cosmos DB, melhorando assim o desempenho global do sistema e diminuindo a latência de resposta aos pedidos REST.

A camada de funções serverless integra um conjunto de Azure Functions que permitem a execução automática de tarefas específicas sem intervenção manual. Foram implementadas as seguintes funções principais:

- `CommentSentimentFunction`: uma função CosmosDB Trigger que é ativada sempre que um novo comentário é inserido. Esta função utiliza o Language Service baseado no Azure Cognitive Services para realizar análise de sentimento, permitindo classificar os comentários de forma automática.
- `CloseAuctionFunction`: uma função TimerTrigger que é executada periodicamente para encerrar leilões cuja data de fecho tenha expirado.
- `CommentGarbageCollectorFunction`: outra função TimerTrigger, responsável por eliminar periodicamente comentários antigos ou inconsistentes, assegurando a limpeza e coerência dos dados.
- `BlobReplicationFunction`: uma função BlobTrigger que executa a geo-replicação manual do Blob Storage, copiando automaticamente os ficheiros para outra região, complementando assim a replicação automática do Cosmos DB.

Estas funções trabalham em conjunto para manter a integridade e o bom funcionamento do sistema, distribuindo tarefas que de outra forma exigiriam intervenção direta do servidor principal.

Por fim, a camada de gestão de recursos, implementada na classe `AzureManagement`, permite automatizar a criação e configuração de todos os recursos necessários no Azure, evitando procedimentos manuais. Esta classe cria e associa, de forma programática, os componentes necessários, como o Cosmos DB, Redis Cache, Blob Storage e Function Apps, além de gerar automaticamente os ficheiros de configuração com as chaves de acesso e endpoints.

A comunicação entre estas camadas segue um fluxo definido: o servidor REST, alojado no App Service, recebe os pedidos dos clientes e verifica primeiro a existência dos dados na cache Redis. Em caso de falha de cache, a informação é obtida a partir do Cosmos DB e guardada temporariamente em cache para otimizar futuras requisições. Os conteúdos multimédia são armazenados no Blob Storage, e as Azure Functions executam tarefas assíncronas associadas ao ciclo de vida das entidades.

## **Implementação**

O projeto foi inteiramente desenvolvido em Java, recorrendo às bibliotecas oficiais do Azure SDK for Java. Foram definidas e implementadas classes que correspondem diretamente às entidades do sistema e aos serviços que as manipulam.

A classe `CosmosDBLayer` desempenha o papel central na interação com a base de dados. Esta classe inicializa os containers correspondentes a cada entidade e fornece métodos para todas as operações CRUD. Adicionalmente, implementa mecanismos de sincronização com a cache Redis, garantindo que os dados armazenados localmente não

entram em conflito com os existentes no Cosmos DB. A camada de persistência também comunica diretamente com as Azure Functions através de bindings automáticos, permitindo acionar eventos quando são detetadas alterações nas coleções de dados.

A classe RedisCache cria e gere a ligação ao serviço Redis, utilizando connection pooling para otimizar o acesso concorrente. Implementa também métodos auxiliares para armazenar, consultar e eliminar dados em cache, além de validar sessões de utilizadores. Complementarmente, a classe CacheHelper disponibiliza funções genéricas de serialização e manipulação de listas em cache, simplificando a integração com o resto do sistema.

As Azure Functions desempenham um papel fundamental na automatização de processos internos. A CommentSentimentFunction analisa o texto de novos comentários e atualiza automaticamente a classificação de sentimento associada ao Lego Set. A CloseAuctionFunction encerra leilões expirados e ajusta o estado da venda. A CommentGarbageCollectorFunction executa limpezas periódicas de comentários antigos, e a BlobReplicationFunction replica automaticamente os ficheiros multimédia para outra região do Azure, assegurando redundância e tolerância a falhas.

A funcionalidade de Azure Cognitive Search foi integrada para permitir pesquisas semânticas sobre os Lego Sets, analisando o conteúdo textual das descrições e comentários. O serviço devolve um conjunto de Lego Sets relacionados com os termos pesquisados pelo utilizador, aumentando a relevância e usabilidade do sistema.

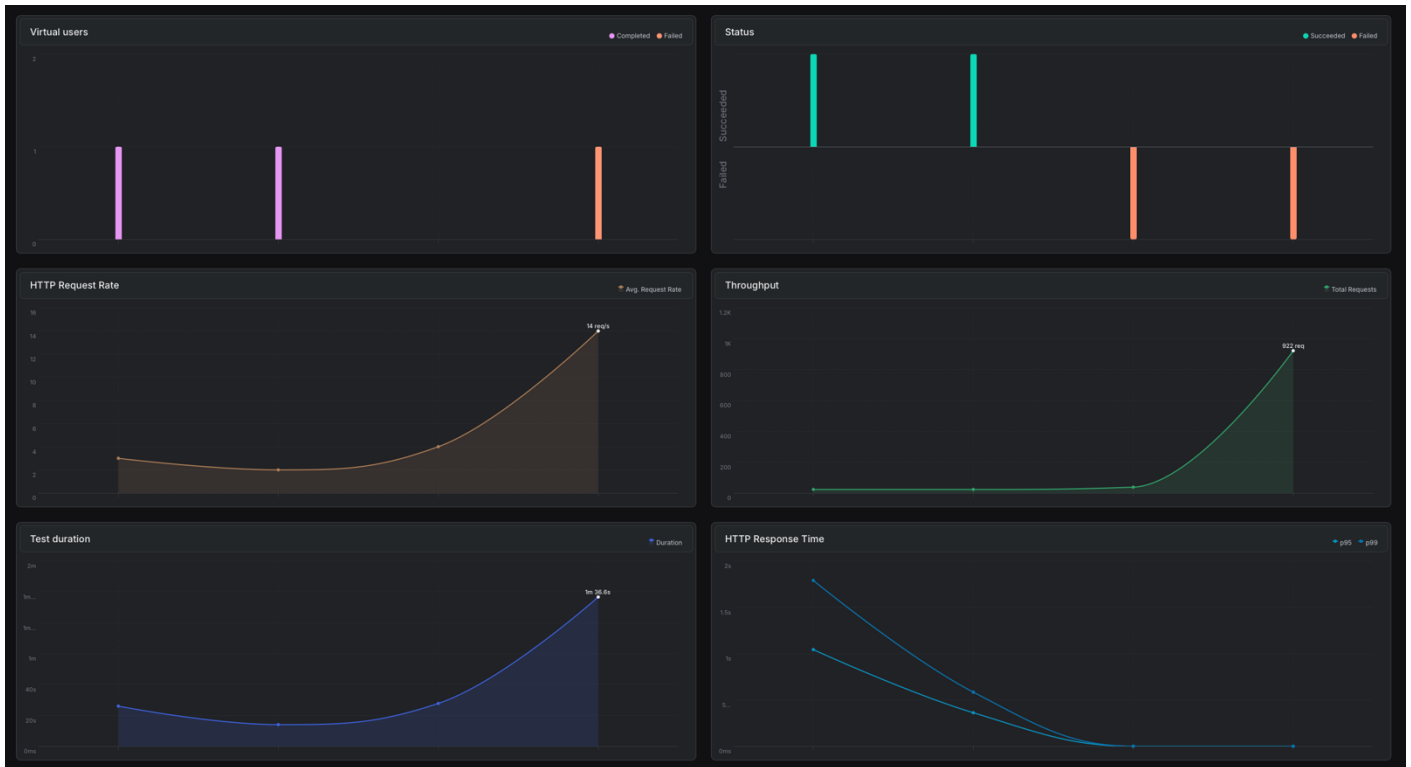
Por fim, a classe AzureManagement automatiza a criação dos recursos necessários no Azure, como grupos de recursos, contas de armazenamento, bases de dados Cosmos e instâncias Redis, gerando ainda os ficheiros de propriedades com as credenciais de acesso. Este processo elimina a necessidade de configurações manuais e garante consistência entre ambientes de desenvolvimento e produção.

## **Avaliação**

A avaliação do desempenho do sistema foi realizada utilizando a ferramenta Artillery, com o objetivo de analisar o comportamento da aplicação sob diferentes condições de carga e configuração. Seguindo as orientações do projeto, foram testados três cenários distintos:

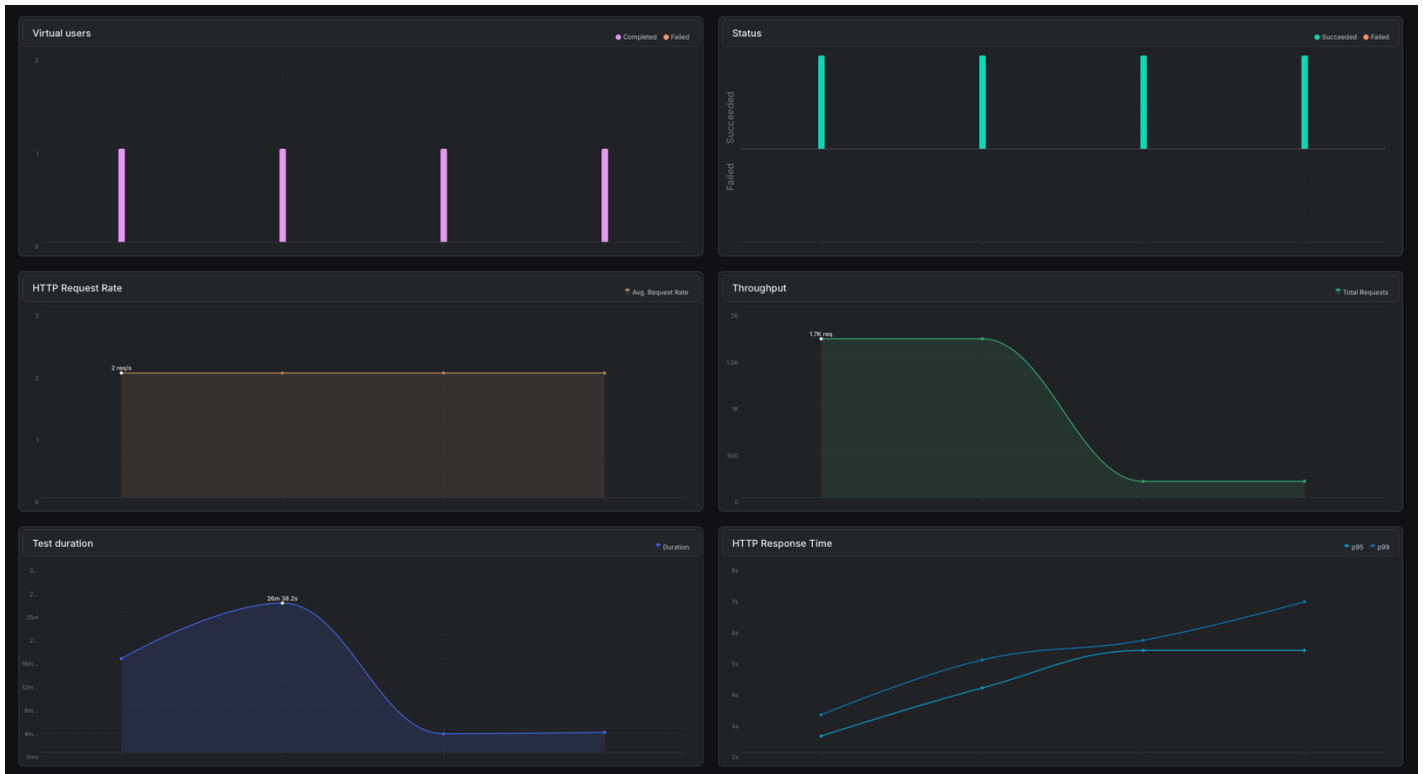
- (a) Aplicação implantada na região Spain Central, com cache ativa;
- (b) Aplicação implantada na região Spain Central, sem cache;
- (c) Aplicação implantada numa segunda região, Norway East, geograficamente distante, com e sem cache.

Nos testes do cenário (a), observou-se o melhor desempenho global. O uso da Azure Cache for Redis permitiu uma redução significativa na latência média, melhorando a rapidez de resposta nas operações de leitura e diminuindo a carga sobre a base de dados Cosmos DB. O sistema mostrou-se capaz de manter tempos de resposta estáveis mesmo sob cargas mais elevadas, evidenciando um throughput superior e uma utilização mais eficiente dos recursos de rede e processamento.

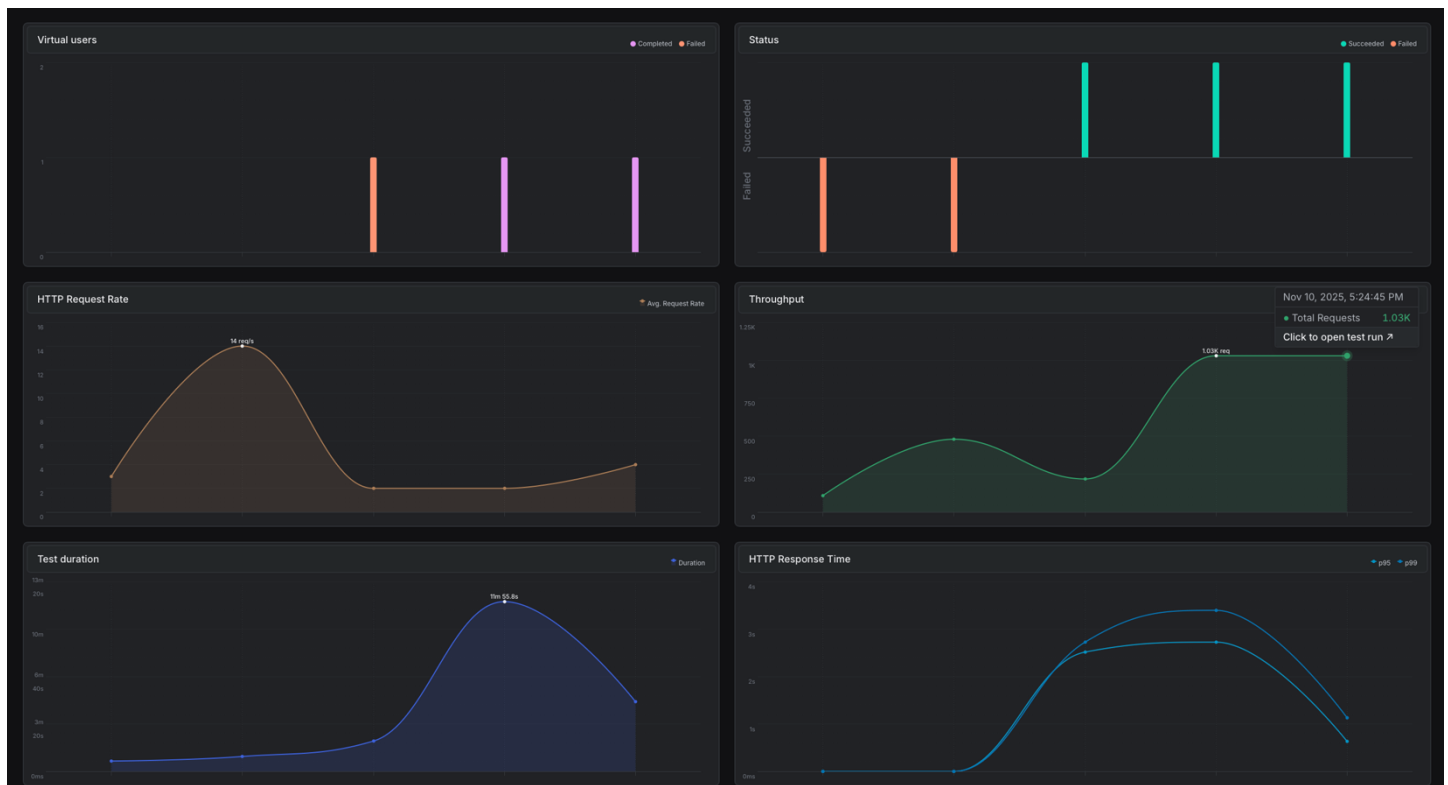


No cenário (b), correspondente à execução na mesma região, mas sem cache, verificou-se um aumento substancial da latência e uma maior variabilidade nos tempos de resposta. A ausência de cache levou a um maior número de acessos diretos ao Cosmos DB, resultando em períodos de maior espera e em picos de utilização de recursos. Apesar

disso, o sistema manteve estabilidade funcional e respondeu corretamente a todas as requisições.



O cenário (c), em que a aplicação foi executada na região Norway East, demonstrou o impacto da distância geográfica na comunicação entre o cliente e o servidor. Mesmo com o uso de cache, os tempos de resposta foram naturalmente superiores aos registados na Spain Central, embora dentro de valores aceitáveis. Quando a cache foi desativada neste contexto, o aumento de latência tornou-se mais pronunciado, mas sem comprometer a fiabilidade do sistema.



Em síntese, os resultados confirmam que o sistema apresenta melhor desempenho e estabilidade quando a cache está ativa, independentemente da região de execução. A análise comparativa entre os três cenários evidencia a relevância da cache Redis na otimização do tempo de resposta e na redução da carga sobre o Cosmos DB. As Azure Functions, executadas em paralelo, não introduziram atrasos significativos, validando a eficiência da abordagem serverless adotada.

## Conclusões

O desenvolvimento deste projeto permitiu compreender de forma prática o potencial da computação em nuvem para a construção de aplicações modernas, escaláveis e resilientes. A utilização coordenada dos serviços do Microsoft Azure, nomeadamente o Cosmos DB, Redis Cache, Blob Storage e Azure Functions, revelou-se eficaz para garantir simultaneamente desempenho e consistência dos dados.

A integração de mecanismos de cache reduziu substancialmente a carga sobre a base de dados, enquanto o uso de funções serverless automatizou processos essenciais, como o encerramento de leilões e a análise de sentimentos. A automação da criação dos recursos de infraestrutura reforçou a reprodutibilidade do ambiente e simplificou a gestão do sistema.

Em síntese, o sistema cumpre integralmente os requisitos obrigatórios estabelecidos no enunciado do projeto: implementa os serviços REST necessários, incorpora cache e Azure Functions, e inclui uma avaliação comparativa do desempenho com e sem cache.

O projeto constitui, assim, um exemplo completo de aplicação cloud nativa, que demonstra o valor da integração entre serviços distribuídos, automação e otimização de recursos em ambiente Azure.

Trabalho realizado por:

Vicente Ribeiro - 72990 - [vg.ribeiro@campus.fct.unl.pt](mailto:vg.ribeiro@campus.fct.unl.pt)

Miguel Teixeira - 72922 - [mre.teixeira@campus.fct.unl.pt](mailto:mre.teixeira@campus.fct.unl.pt)

Madalena Alves - 75150 - [mfo.alves@campus.fct.unl.pt](mailto:mfo.alves@campus.fct.unl.pt)